

Quebrando o anonimato de assinaturas em anel Ed25519

Desafio *Put a ring on it* (ECSC 2023, CryptoHack) - reuso de estado e vazamento de chave

Universidade Federal de São Carlos - Disciplina de CyberSegurança

4 de setembro de 2026

Tema: **Criptografia**

- 1 O desafio e a motivação: anonimato em assinaturas em anel
- 2 Fundamentos: curvas elípticas e Ed25519
- 3 O “ciclo” da assinatura em anel
- 4 Visualização do valor q ao longo do anel
- 5 A falha: estado indevidamente reutilizado
- 6 O ataque (oráculo) e o resultado
- 7 Lições aprendidas e referências

Por que assinaturas em anel?

- Usadas em criptomoedas (Monero / CryptoNote) para ocultar *quem* assinou/transferiu.
- Um grupo de chaves públicas forma o “anel”; qualquer verificador prova que *alguém do anel* assinou - sem descobrir quem.

Arquivos do desafio

- ed25519.py - curva elíptica (Ed25519)
- chal.py - gera 16 “níveis” com assinaturas
- data.json - 16 assinaturas + chaves públicas + flag cifrada

Nosso objetivo

- Descobrir, em cada nível, **qual chave pública** realmente assinou (o índice `my_key`, removido de `data.json`).
- Com isso, montar a chave AES e decifrar a flag.

Pista

A “imagem de chave” (`key_image`) é a mesma para o mesmo signatário $\Rightarrow$ serve de *oráculo* para confirmar um candidato.

- Curva de Edwards: $-x^2 + y^2 = 1 + dx^2y^2$ sobre $\mathbb{F}_q$, com $q = 2^{255} - 19$.
- Ponto gerador B de ordem prima ℓ ; escalares em $\mathbb{Z}_\ell$.
- **Privada** $x \in_R \mathbb{Z}_\ell$; **pública** $P = xB$; recuperar x de P é o problema do logaritmo discreto (inviável).
- **Hash-para-ponto** $H_p(U) = hB$, com $h = H(U)$ (ex.: SHA-256): mapeia textos/pontos na curva sem revelar o “logaritmo” de $H_p(U)$.

`random_scalar()` = 256 bits aleatórios (privada x)
`public_key(sk)` = $P = sk \cdot B$ (ponto comprimido)

Assinaturas em anel: conceito e garantias

Configuração

- $n = 16$ chaves $P_0, \dots, P_{15}$ (o anel); o signatário real, no índice π , conhece x com $P_\pi = xB$; as demais 15 são o “disfarce”.

Imagem de chave

$$I = x \cdot H_p(P_\pi)$$

- Só quem conhece x computa I ; mesmo signatário $\Rightarrow$ mesma imagem (detecta gasto duplo sem revelar quem).

Propriedades esperadas

- 1 **Autenticidade:** prova-se que algum membro do anel assinou.
- 2 **Anonimato:** não se descobre qual membro assinou.
- 3 **Não falsificabilidade:** só quem tem x assina.

O desafio ataca a propriedade (2): o anonimato.

O “ciclo” da assinatura em anel (1/2): percorrer os elos

Percorremos o anel com $i = 0, 1, \dots, 15$; em cada *elo* calculamos um par (L_i, R_i) :

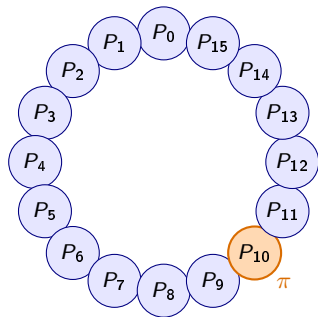
Elo “falso” ($i \neq \pi$): sorteia c_i e r_i

$$L_i = r_i B + c_i P_i \quad R_i = r_i H_p(P_i) + c_i I$$

guarda $\text{sigc}[i] = c_i$ e $\text{sigr}[i] = r_i$. **Elo do signatário**

($i = \pi$): ainda não conhece seu desafio c_π ; reserva um **segredo** α :

$$L_\pi = \alpha B \quad R_\pi = \alpha H_p(P_\pi)$$



Anel de 16
chaves; laranja = P_π (signatário).

O “ciclo” da assinatura em anel (2/2): fechar o anel

1. **Desafio global** - hash de todo o percurso do anel:

$$c = H(\text{msg} \parallel L_0 \parallel R_0 \parallel \cdots \parallel L_{15} \parallel R_{15})$$

2. **Fechamento** - o desafio do signatário completa a soma dos desafios:

$$c_\pi = \left(c - \sum_{i \neq \pi} c_i \right) \bmod \ell \quad \Rightarrow \quad \sum_i c_i \equiv c \pmod{\ell}$$

3. **Resposta** - o segredo α esconde a chave x : $r_\pi = (\alpha - c_\pi x) \bmod \ell$.
4. **Assinatura**: $(I; c_0, \dots, c_{15}; r_0, \dots, r_{15})$.

Verificação (ideia)

Recalcula $L_i = r_i B + c_i P_i$ e $R_i = r_i H_p(P_i) + c_i I$ para todo i e confere $\sum_i c_i \equiv H(\cdots)$. Sem x não existe r_π coerente; para o verificador, todos os candidatos π são indistinguíveis.

Visualização: o valor q ao longo dos elos

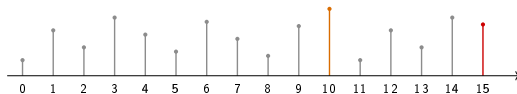
Na implementação, o “segredo por elo” é a variável q .

- A cada iteração i **sorteia-se um q novo**.
- Elos falsos: q_i vira a resposta r_i e é *guardado* em $\text{sigr}[i]$.
- No elo do signatário ($i = \pi$), $q = \alpha$ é usado em L_π, R_π e deveria permanecer *secreto* para $r_\pi = \alpha - c_\pi x$.

α : sorteado no elo 10 e usado em L_π, R_π

q_{15} : valor que “sobra” em q

demais elos: cada um tem seu q_i (disfarce)



Valores ilustrativos - o ponto é que cada elo sorteia um q novo.

Atenção

Se α vazar, x é recuperado: $x = (\alpha - r_\pi) c_\pi^{-1}$.

A falha: estado indevidamente reutilizado

Essência de chal.py (linhas $\approx$ 73–105, simplificada):

```
for i in range(RING_SIZE):
    if i == my_index:
        q = random_scalar()
        L = q*B
        R = q*Hp(P_pi)
    else:
        q = random_scalar()
        w = random_scalar()
        L = q*B + w*P_i
        R = q*Hp(P_i) + w*I
        sigc[i], sigr[i] = w, q
    buf += L + R
c = scalar_hash(buf)
sigc[my_index] = (c - sumc) % ed25519.1
sigr[my_index] = (q - sigc[my_index]*my_privkey) % ed25519.1
```

elo do signatario (pi)
alpha
*# L_pi = alpha*B*
*# R_pi = alpha*Hp(P_pi)*
elo falso (disfarce)
r_i
c_i
*# r_i*B + c_i*P_i*
*# r_i*Hp(P_i) + c_i*I*
c_i e r_i vao para a assinatura
desafio global

- q é **sobrescrita** a cada elo; o α do signatário **nunca é salvo** e, ao fim do laço, q vale o último sorteio - q_{15} , já publicado em `sigr[15]`.

A matemática do vazamento

Para um elo *falso*, $r_i = q_i$ é aleatório: não há relação algébrica com as chaves. Já para o signatário (quando $\pi \neq 15$):

$$\text{sigr}[\pi] = (q_{15} - c_\pi x) \bmod \ell \quad \text{com} \quad q_{15} = \text{sigr}[15].$$

Como q_{15} , c_π e $\text{sigr}[\pi]$ estão **todos na assinatura pública**, para qualquer candidato i :

$$x_i = (\text{sigr}[15] - \text{sigr}[i]) \cdot \text{sigc}[i]^{-1} \pmod{\ell}$$

só produz a chave *verdadeira* quando $i = \pi$.

Analogia clássica

Mesmo princípio do **reuso de nonce** em Schnorr/ECDSA: se o nonce (α) torna-se conhecido, a equação da assinatura fica linear e revela a chave. Aqui α foi trocado por um valor já publicado no próprio anel.

Consequência: anonimato quebrado

- A assinatura carrega uma equação *resolvível*: testamos candidatos e conferimos com a imagem de chave I .
- **Descobre-se quem assinou** - e a chave privada x vaza.
- Caso $\pi = 15$: ao fim do laço q é o próprio α ; a equação não recupera x , mas o índice 15 é identificado por **eliminação** (nenhum dos 15 primeiros candidatos confere).
- *Curiosidade*: com $\pi \neq 15$ a assinatura nem “fecha o anel” na verificação (o r_π não corresponde ao α de L_π); o gerador nunca verifica - e os dados ainda bastam.

$\pi \neq 15$	x vaza diretamente (15 em 16 níveis)
$\pi = 15$	índice identificado por eliminação

O ataque: oráculo de de-anonimização

Essência do solve.py:

```
for l in data["levels"]:                                # para cada um dos 16 niveis
    image, sigc, sigr = l["signature"]
    q = sigr[15]                                         # q15 esta publico em sigr
    for i in range(15):                                  # testa candidatos 0..14
        priv = (q - sigr[i]) * inverse(sigc[i], ed25519.1) % ed25519.1
        if image == key_image(priv):                    # confirma com a imagem de chave
            my_keys.append(i)
            break
    else:
        my_keys.append(15)                              # senao: signatario e o elo 15
```

- `key_image(priv)` recomputa $x \cdot H_p(P(x))$; casar com a imagem dada / **confirma** o candidato.
- Nenhum candidato em $0, \dots, 14$ confere $\Rightarrow$ signatário é o elo 15.

Aplicando nos 16 níveis e montando a chave AES

Montagem da chave

- Chave AES = 1º byte (2 hex) da chave pública *verdadeira* de cada nível.
- 16 níveis $\Rightarrow$ 16 bytes $\Rightarrow$ AES-128-CBC.

```
key = ''.join(pks[my_keys[lvl]][:2]
              for lvl, pks in enumerate(...))
AES.new(bytes.fromhex(key),
        AES.MODE_CBC, iv).decrypt(enc)
```

Índices recuperados (conferem com o gabarito):

0:10	1:6	2:12	3:5	4:14	5:12	6:4	7:5
8:6	9:14	10:14	11:0	12:2	13:11	14:6	15:4

Validação da reprodução

- chal.py gera data-private.json (gabarito com my_key) e data.json (público, sem my_key).
- O oráculo recuperou **16 de 16** índices corretos.

Resultado

- chave AES (local):
3549b93a989cc715e68eb5e718be7755
- flag (reprodução local):
ECSC{fake_flag_}
- flag original do desafio:
ECSC{thx_Vinc0682_4_inspiration}

Execução: logs da implementação

Ataque - python solve.py (execução real):

```
$ python solve.py
[+] data.json: 16 níveis + flag cifrada (AES-128-CBC)

-- nivel 1/16 --
q15 = sigr[15]      : 571b863136... (publico!)
candidato 0: x = c8acbc1cf8... | key_image == I ? nao
    casa
... (testa 0..10)
candidato 10: x = e57ffe9e78... | key_image == I ?
    CONFIRMA
==> signatario = indice 10 (11 testes, 11.2s)
... 16/16 níveis conferem com o gabarito
[+] índices recuperados:
    [10,6,12,5,14,12,4,5,6,14,14,0,2,11,6,4]
[+] chave AES (hex) : 3549b93a989cc715e68eb5e718be7755
[+] tempo total : 144.6s
[+] flag : ECSC{fake_flag_}
```

Geração - chal.py -trace-demo (um anel):

```
my_index = 4
[trace] elo 3: q = r_3 = cd131abcf5... ->
    sigr[3]=q
[trace] elo 4: q = alpha = ab465d39ca...
    (signatario; NAO e salvo)
[trace] elo 15: q = r_15 = 8b2c940854... ->
    sigr[15]=q
[trace] fim: q = 8b2c940854... = q_15 (
    publico!)
[trace] sigr[4] usa ESSE q publico -> vaza x
    !
```

Não altera data.json; flag local é fictícia.

Como reproduzir (passo a passo)

- 1 `python chal.py` - gera `data-private.json` (com `my_key`) e `data.json` (público, sem `my_key`), cifrando o conteúdo de `flag.txt`.
- 2 `python solve.py` - roda o oráculo e decifra a flag.
- 3 (Opcional) conferir com o gabarito em `data-private.json`.

<code>data.json</code>	→	ataque (anônimo)	→	flag
<code>data-private.json</code>	→	gabarito (só para validar)		

O ataque usa apenas `data.json`: nunca recalculamos o hash/verificação - só exploramos a relação linear entre `sigr[15]`, `sigr[ $\pi$ ]` e `sigc[ $\pi$ ]`.

- **Segredos efêmeros (nonces) são sagrados:** reutilizar ou vazar α destrói assinaturas do tipo Schnorr - em anel ou não.
- **Cuidado com estado de controle:** a variável q reutilizada no laço virou “estado” que alimentou a criptografia; valores que “sobram” não devem influenciar operações críticas.
- **Implementar cripto na mão é arriscado:** prefira bibliotecas auditadas (ex.: MLSAG do Monero) e *sempre verifique* a assinatura.

- CryptoHack - *CTF Archive: Put a ring on it (ECSC 2023)*: https://github.com/crytohack/ctf_archive/tree/main/ecsc2023-put-a-ring-on-it
- Write-up de 7Rocky:
<https://7rocky.github.io/en/ctf/other/ecsc-2023/put-a-ring-on-it/>
- Desafio no CryptoHack: <https://crytohack.org/challenges/ctf-archive/>
- ed25519.py do mininero (Monero): <https://github.com/monero-project/mininero>
- RFC 8032 (Ed25519); Wikipedia: *Ring signature*; Monero *MLSAG*.

Obrigado!

Perguntas?